Jack Sevigny, Jack Berg

1/14/2026

A2

Demo Link:

https://youtube.com/shorts/wHTyc4wKI3w?feature=share

Main.c

```
/**
 * @file    main.c
 * @brief   Application entry point for keypad-to-LED display.
 *
 * This file contains the main application loop. The program initializes
 * the HAL, LED GPIO hardware, and keypad GPIO hardware. It then polls the
 * keypad and displays the detected key value on the LEDs.
 *
 * Modules used:
 *   - keypad: Scans a 3x4 matrix keypad and returns an int16_t key code.
 *   - leds  : Drives an LED display and renders an int16_t value.
 *
 * Notes:
 *   - The keypad is polled in the while(1) loop.
 *   - A short delay is inserted after a valid key press to reduce bounce
 *     and repeated detections.
 *
 * Dependencies:
 *   - STM32 HAL initialization (HAL_Init)
 *   - keypad_init(), keypad_getkey()
 *   - led_init(), display_on_leds()
 *
```

```
 * @author  Jack Sevigny
 * @date    2026-01-15
 * (chat-gpt formatted header)
 */

#include "main.h"
#include "keypad.h"
#include "leds.h"
#include <stdint.h>

#define KEY_DEBOUNCE_DELAY_MS (200u)  //Key debounce/repeat delay (ms).

int main(void)
{
  int16_t key_code = (int16_t)-1;

  HAL_Init();
  led_init();
  keypad_init();

  while (1) {
    key_code = keypad_getkey();
    if (key_code != (int16_t)-1) {
      display_on_leds(key_code);
      HAL_Delay(KEY_DEBOUNCE_DELAY_MS); // delay
    }
  }
}
```

Keypad.c

```
/**
 * @file    keypad.c
 * @brief   3x4 matrix keypad driver (GPIO scan).
 *
 * This module configures and scans a 3-row-by-4-column style keypad
 * wired as a matrix to STM32L4 GPIO port B.
 *
 * Wiring (as used by this driver):
 *   Rows (inputs w/ pull-down):  PB0, PB1, PB2, PB7
 *   Cols (outputs, driven high): PB4, PB5, PB6
 *
 * Operation:
 *   1) Drive all columns high and check if any row reads high.
 *   2) Drive one column high at a time to identify the pressed key.
 *
 * Public functions:
 *   - keypad_init()   : Configure GPIO pins for keypad scanning
 *   - keypad_getkey() : Return key code or -1 if no key is pressed
 *
 * Notes:
 *   - This is a polled (non-interrupt) driver.
 *   - A short settle delay is used after driving a column.
 *   (chatgpt formatted header and file)
 */
```

```
#include "keypad.h"
#include "stm32l4xx.h"
#include <stdint.h>

#define NUM_ROWS            (4u)
#define NUM_COLS            (3u)

#define ROW_PORT            (GPIOB)
#define COL_PORT            (GPIOB)

// Row inputs: PB0, PB1, PB2, PB7
static const uint8_t row_pin[NUM_ROWS] = { 0u, 1u, 2u, 7u };
#define ROW_PINS_MASK       ((1u << 0) | (1u << 1) | (1u << 2) | (1u << 7))

// Column outputs: PB4..PB6
#define COL_OFFSET          (4u)    // PB4 is column 0
#define COL_MASK            ((((1u << NUM_COLS) - 1u) << COL_OFFSET))

// Scan timing
#define COL_SETTLE_DELAY_CYC    (200u)  // NOP cycles after driving a column

#define KEY_NONE            ((int16_t)-1)

// Key codes for [row][col]
static const int16_t keymap[NUM_ROWS][NUM_COLS] = {
  {  1,  2,  3 },
```

```
  { 4, 5, 6 },
  { 7, 8, 9 },
  { 10, 0, 11 }
};

static void delay_cycles(uint32_t cycles)
{
  volatile uint32_t idx = 0u;

  for (idx = 0u; idx < cycles; idx++) {
    __NOP();
  }
}

static void cols_all_low(void)
{
  // Reset PB4..PB6 (atomic)
  COL_PORT->BSRR = (COL_MASK << 16);
}

static void cols_all_high(void)
{
  // Set PB4..PB6 (atomic)
  COL_PORT->BSRR = COL_MASK;
}
```

```
static void col_one_high(uint8_t col_idx)
{
   cols_all_low();
   COL_PORT->BSRR = (1u << (COL_OFFSET + col_idx));
}

void keypad_init(void)
{
   uint32_t idx = 0u;
   uint32_t pin = 0u;

   RCC->AHB2ENR |= RCC_AHB2ENR_GPIOBEN;

   // Configure row pins as input with pull-down.
   for (idx = 0u; idx < NUM_ROWS; idx++) {
      pin = (uint32_t)row_pin[idx];

      ROW_PORT->MODER &= ~(3u << (2u * pin));    // input
      ROW_PORT->PUPDR &= ~(3u << (2u * pin));
      ROW_PORT->PUPDR |=  (2u << (2u * pin));    // pull-down
   }

   // Configure column pins as push-pull outputs, low speed, no pull.
   for (idx = 0u; idx < NUM_COLS; idx++) {
      pin = (uint32_t)COL_OFFSET + idx;
```

```
    COL_PORT->MODER &= ~(3u << (2u * pin));
    COL_PORT->MODER |=  (1u << (2u * pin));    // output

    COL_PORT->OTYPER  &= ~(1u << pin);        // push-pull
    COL_PORT->OSPEEDR &= ~(3u << (2u * pin));  // low speed
    COL_PORT->PUPDR   &= ~(3u << (2u * pin));  // no pull
  }

  cols_all_low();
}

int16_t keypad_getkey(void)
{
  uint32_t rows = 0u;
  uint32_t r = 0u;
  uint32_t c = 0u;

  // Stage 1: detect any key press (any row high with all cols high).
  cols_all_high();
  delay_cycles(COL_SETTLE_DELAY_CYC);

  rows = ROW_PORT->IDR & ROW_PINS_MASK;
  if (rows == 0u) {
    cols_all_low();
    return KEY_NONE;
  }
```

```
  // Stage 2: identify key by scanning each column.
  for (c = 0u; c < NUM_COLS; c++) {
    col_one_high((uint8_t)c);
    delay_cycles(COL_SETTLE_DELAY_CYC);

    rows = ROW_PORT->IDR & ROW_PINS_MASK;
    if (rows != 0u) {
      for (r = 0u; r < NUM_ROWS; r++) {
        if ((rows & (1u << row_pin[r])) != 0u) {
          cols_all_low();
          return keymap[r][c];
        }
      }
    }
  }

  cols_all_low();
  return KEY_NONE;
}
```

Leds.c

```
/**
 * @file    leds.c
 * @brief   LED control interface.
 *
 * This module provides initialization and display functions for a
 * 4-bit LED output implemented using GPIO pins PC0–PC2 and PA5.
 *
 * The LEDs are used to display a signed 16-bit value, typically
 * originating from keypad input or other application logic.
 *
 * Public functions:
 *   - led_init()       : Configure GPIO pins for LED output
 *   - display_on_leds()  : Display a 4-bit value on the LEDs
 *
 * Hardware mapping:
 *   - PC0 : LED bit 0 (LSB)
 *   - PC1 : LED bit 1
 *   - PC2 : LED bit 2
 *   - PA5 : LED bit 3 (MSB / onboard LED)
 *   (chat-gpt formatted header and file layout)
 */
#include "leds.h"
#include "stm32l4xx.h"
#include <stdint.h>
```

```
void led_init(void) {



    // Enable GPIOA and GPIOC clocks

    RCC->AHB2ENR |= (RCC_AHB2ENR_GPIOAEN | RCC_AHB2ENR_GPIOCEN);


    // ---- PC0, PC1, PC2 as outputs (01) ----

    GPIOC->MODER &= ~(GPIO_MODER_MODE0 | GPIO_MODER_MODE1 |
GPIO_MODER_MODE2);

    GPIOC->MODER |=  (GPIO_MODER_MODE0_0 | GPIO_MODER_MODE1_0 |
GPIO_MODER_MODE2_0);


    GPIOA->MODER &= ~(GPIO_MODER_MODE5);

    GPIOA->MODER |=  (GPIO_MODER_MODE5_0);

    GPIOC->OTYPER &= ~(GPIO_OTYPER_OT0 | GPIO_OTYPER_OT1 |
GPIO_OTYPER_OT2); // push-pull

    GPIOC->PUPDR  &= ~(GPIO_PUPDR_PUPD0 | GPIO_PUPDR_PUPD1 |
GPIO_PUPDR_PUPD2); // no pull


    // ---- PA5 as output (01) ---

    GPIOA->OTYPER &= ~(GPIO_OTYPER_OT5);

    GPIOA->PUPDR  &= ~(GPIO_PUPDR_PUPD5);
```

```
}
void display_on_leds(int16_t count) {
        // Clear PC0..PC2, PA5 then set bits based on input
            uint16_t u = (uint16_t)count;
        GPIOC->ODR &= ~(GPIO_ODR_OD0 | GPIO_ODR_OD1 | GPIO_ODR_OD2);
        GPIOA->ODR &= ~(GPIO_ODR_OD5);
        if (u == 0) {
            GPIOC->ODR &= ~(GPIO_ODR_OD0);
            GPIOA->ODR &= ~(GPIO_ODR_OD5);
        }
        if (u & (1u << 0)) GPIOC->ODR |= GPIO_ODR_OD0;
        if (u & (1u << 1)) GPIOC->ODR |= GPIO_ODR_OD1;
        if (u & (1u << 2)) GPIOC->ODR |= GPIO_ODR_OD2;
        if (u & (1u << 3)) GPIOA->ODR |= GPIO_ODR_OD5;// set pin 5 of PORT A to 1 for
onboard led
    }
```